

Bank Management System

Problem Statement

To design and implement a **Bank Management System using Java** that allows users to create bank accounts, perform ATM operations such as deposit, withdrawal, money transfer, and view account statements. The system ensures basic security using PIN authentication and restricts access so that users can only view and manage their own accounts, while the admin can monitor all accounts.

Concepts Used

Concept	Purpose
Classes & Objects	Represent bank accounts and system operations
Methods	Perform operations like deposit, withdraw, transfer
Arrays	Store multiple account records
ArrayList	Maintain transaction history
Strings	Store names, account numbers, and transaction logs
Conditional Statements	Validate PIN, balance checks, and account access
Loops	Run menu system and iterate through accounts
Encapsulation	Protect account data like PIN and balance

Short Description of Working

1. The system starts with a **main menu** that allows users to create accounts or access banking services.
2. A new user must **create an account** by providing personal details and setting a secure PIN.
3. Existing users must **log in using account number and PIN** to perform operations.
4. Users can perform banking operations such as:
 - Deposit money
 - Withdraw money
 - Transfer money to another account
 - View account details and mini statement
5. Every transaction is recorded in a **transaction history list**.
6. An **admin panel** allows authorized staff to view all accounts in the system.
7. The program runs in a **console-based interactive menu system** until the user exits.

BankingApplication.java

```
1  /**
2   *Project: Simple Bank Management System.
3   *      It allows users to create accounts, deposit or withdraw money,
4   * transfer funds, and view their transactions.
5   * Access is protected using a PIN, and an admin panel can view all accounts.
6   * |This project demonstrates basic OOP concepts, arrays, methods, and console-based menus.
7   **/
8
9  import java.util.*;
10
11  /* Account class stores all account information and transactions */
12  class Account { 19 usages
13
14      int accountNumber; 5 usages
15      String name; 2 usages
16      String phone; 3 usages
17      int pin; 2 usages
18      double balance; 6 usages
19
20      ArrayList<String> transactions = new ArrayList<>(); 5 usages
21
22      /* Constructor creates a new bank account */
23      Account(int accountNumber, String name, String phone, int pin, double balance) { 1 usage
24          this.accountNumber = accountNumber;
25          this.name = name;
26          this.phone = phone;
27          this.pin = pin;
28          this.balance = balance;
29
30          transactions.add("Account created with balance: " + balance);
31      }
32
33      /* Deposit money into account */
34      void deposit(double amount) { 2 usages
35          balance += amount;
36          transactions.add("Deposited: " + amount);
37      }
38  }
```

```
39      /* Withdraw money if sufficient balance */
40      boolean withdraw(double amount) { 2 usages
41          if (amount > balance) {
42              return false;
43          }
44
45          balance -= amount;
46          transactions.add("Withdrawn: " + amount);
47          return true;
48      }
49
50      /* Add transfer transaction record */
51      void addTransaction(String message) { 2 usages
52          transactions.add(message);
53      }
54
55      /* Print account details */
56      void showDetails() { 2 usages
57          System.out.println("\n==== ACCOUNT DETAILS =====");
58          System.out.println("Account Number : " + accountNumber);
59          System.out.println("Name           : " + name);
60          System.out.println("Phone          : " + phone);
61          System.out.println("Balance         : " + balance);
62      }
63
64      /* Print account statement */
65      void showStatement() { 2 usages
66          System.out.println("\n==== MINI STATEMENT =====");
67
68          for (String t : transactions) {
69              System.out.println(t);
70          }
71      }
72  }
73
```

```

74
75  /* Main banking system class which controls all operations */
76  class BankSystem { 12 usages
77
78      static Scanner sc = new Scanner(System.in); 14 usages
79
80      /* Array storing all bank accounts */
81      static Account accounts[] = new Account[100]; 4 usages
82      static int accountCount = 0; 4 usages
83
84      /* Admin credentials */
85      static String adminUser = "admin"; 1 usage
86      static String adminPass = "bank123"; 1 usage
87
88      static int nextAccountNumber = 1001; 3 usages
89
90      /* Method to create a new account */
91      static void createAccount() { 1 usage
92
93          System.out.println("\n==== CREATE ACCOUNT =====");
94
95          System.out.print("Enter Name: ");
96          String name = sc.next();
97
98          System.out.print("Enter Phone: ");
99          String phone = sc.next();
100
101          System.out.print("Set 4 digit PIN: ");
102          int pin = sc.nextInt();
103
104          System.out.print("Initial Deposit: ");
105          double balance = sc.nextDouble();
106
107          Account acc = new Account(nextAccountNumber, name, phone, pin, balance);
108
109          accounts[accountCount] = acc;
110          accountCount++;
111
112          System.out.println("\nAccount Created Successfully!");
113          System.out.println("Your Account Number: " + nextAccountNumber);
114
115          nextAccountNumber++;
116      }
117
118      /* Method to search account using account number */
119      @~ static Account findAccount(int accNo) { 2 usages
120
121          ~ for (int i = 0; i < accountCount; i++) {
122              if (accounts[i].accountNumber == accNo)
123                  return accounts[i];
124          }
125
126          return null;
127      }

```

```

128
129      /* User login using account number and pin */
130      @ static Account login() { 6 usages
131
132          System.out.print("Enter Account Number: ");
133          int accNo = sc.nextInt();
134
135          Account acc = findAccount(accNo);
136
137          if (acc == null) {
138              System.out.println("Account not found");
139              return null;
140          }
141
142          System.out.print("Enter PIN: ");
143          int pin = sc.nextInt();
144
145          if (acc.pin != pin) {
146              System.out.println("Incorrect PIN");
147              return null;
148          }
149
150          return acc;
151      }
152
153      /* Deposit money to account */
154      @ static void depositMoney(Account acc) { 1 usage
155
156          System.out.print("Enter amount to deposit: ");
157          double amt = sc.nextDouble();
158
159          acc.deposit(amt);
160
161          System.out.println("Deposit successful");
162      }
163
164      /* Withdraw money using ATM */
165      @ static void withdrawMoney(Account acc) { 1 usage
166
167          System.out.print("Enter withdrawal amount: ");
168          double amt = sc.nextDouble();
169
170          if (acc.withdraw(amt)) {
171              System.out.println("Withdrawal successful");
172          } else {
173              System.out.println("Insufficient balance");
174          }
175      }
176

```

```

176
177     /* Transfer money between two accounts */
178     static void transferMoney(Account sender) { 1 usage
179
180         System.out.print("Enter receiver account number: ");
181         int receiverNo = sc.nextInt();
182
183         Account receiver = findAccount(receiverNo);
184
185         if (receiver == null) {
186             System.out.println("Receiver account not found");
187             return;
188         }
189
190         System.out.print("Enter transfer amount: ");
191         double amount = sc.nextDouble();
192
193         if (sender.withdraw(amount)) {
194
195             receiver.deposit(amount);
196
197             sender.addTransaction("Transferred " + amount + " to " + receiver.accountNumber);
198             receiver.addTransaction("Received " + amount + " from " + sender.accountNumber);
199
200             System.out.println("Transfer successful");
201         } else {
202             System.out.println("Insufficient balance");
203         }
204     }
205
206     /* Update user details */
207     @ static void updateAccount(Account acc) { 1 usage
208
209         System.out.println("\n==== UPDATE ACCOUNT =====");
210
211         System.out.print("Enter new phone number: ");
212         acc.phone = sc.next();
213
214         System.out.println("Phone updated successfully");
215     }
216

```

```

216
217     /* ATM menu for user */
218     static void atmMenu(Account acc) { 1 usage
219
220         int choice;
221
222         do {
223             System.out.println("\n===== ATM MENU =====");
224
225             System.out.println("1 Withdraw");
226             System.out.println("2 Check Balance");
227             System.out.println("3 Mini Statement");
228             System.out.println("4 Exit ATM");
229
230             choice = sc.nextInt();
231
232             switch (choice) {
233                 case 1:
234                     withdrawMoney(acc);
235                     break;
236                 case 2:
237                     System.out.println("Balance: " + acc.balance);
238                     break;
239                 case 3:
240                     acc.showStatement();
241                     break;
242             }
243
244             } while (choice != 4);
245
246     }

```

```

247
248     /* Admin login to see all accounts */
249     static void adminPanel() { 1 usage
250
251         System.out.print("Admin Username: ");
252         String user = sc.next();
253
254         System.out.print("Admin Password: ");
255         String pass = sc.next();
256
257         if (user.equals(adminUser) && pass.equals(adminPass)) {
258             System.out.println("\n===== ALL ACCOUNTS =====");
259
260             for (int i = 0; i < accountCount; i++) {
261                 accounts[i].showDetails();
262             }
263         } else {
264             System.out.println("Invalid admin login");
265         }
266     }
267 }
268

```

```

269
270  /* Main class containing main menu */
271  ▶ public class BankingApplication {
272
273  ▶      public static void main(String[] args) {
274
275          Scanner sc = new Scanner(System.in);
276          int choice;
277
278          while (true) {
279
280              System.out.println("\n===== BANK MANAGEMENT SYSTEM =====");
281
282              System.out.println("1 Create Account");
283              System.out.println("2 View My Account");
284              System.out.println("3 ATM System");
285              System.out.println("4 Deposit Money");
286              System.out.println("5 Transfer Money");
287              System.out.println("6 Update Account");
288              System.out.println("7 View Statement");
289              System.out.println("8 Admin Panel");
290              System.out.println("9 Exit");
291
292              System.out.print("Enter choice: ");
293              choice = sc.nextInt();
294
295              switch (choice) {
296                  case 1:
297                      BankSystem.createAccount();
298                      break;
299                  case 2: {
300                      Account acc = BankSystem.login();
301                      if (acc != null)
302                          acc.showDetails();
303                      break;
304                  }
305                  case 3: {
306                      Account acc = BankSystem.login();
307                      if (acc != null)
308                          BankSystem.atmMenu(acc);
309                      break;

```

```

310     }
311     case 4: {
312         Account acc = BankSystem.login();
313         if (acc != null)
314             BankSystem.depositMoney(acc);
315         break;
316     }
317     case 5: {
318         Account acc = BankSystem.login();
319         if (acc != null)
320             BankSystem.transferMoney(acc);
321         break;
322     }
323     case 6: {
324         Account acc = BankSystem.login();
325         if (acc != null)
326             BankSystem.updateAccount(acc);
327         break;
328     }
329     case 7: {
330         Account acc = BankSystem.login();
331         if (acc != null)
332             acc.showStatement();
333         break;
334     }
335     case 8:
336         BankSystem.adminPanel();
337         break;
338     case 9:
339         System.out.println("Thank you for using the bank system");
340         System.exit(status: 0);
341     default:
342         System.out.println(("Invalid choice! Please choose between 1-9."));
343         break;
344 }
345 }
346 }
347 }

```

Output:

```
===== BANK MANAGEMENT SYSTEM =====
```

```
1 Create Account
2 View My Account
3 ATM System
4 Deposit Money
5 Transfer Money
6 Update Account
7 View Statement
8 Admin Panel
9 Exit
```

```
Enter choice: 1
```

```
===== CREATE ACCOUNT =====
```

```
Enter Name: John
```

```
Enter Phone: 8794563210
```

```
Set 4 digit PIN: 1234
```

```
Initial Deposit:
```

```
50034
```

```
Account Created Successfully!
```

```
Your Account Number: 1001
```

Console Arcade Game System

Problem Statement

To design and implement a Console Arcade Game System using Java that allows users to select and play multiple games from a single menu interface. The system loads different games such as Tic Tac Toe, Rock Paper Scissors, and Hangman, collects player information, and lets users play games repeatedly through an interactive console menu.

Concepts Used

Concept	Purpose
Classes & Objects	Represent each game and the main arcade system
Packages	Organize multiple game files in a structured project
Methods	Implement game logic and menu operations
Arrays	Store game boards, words, and options
Random Class	Generate random choices and words
Strings	Store player names, guesses, and words
Conditional Statements	Control game rules and outcomes
Loops	Run game rounds and menu system

Short Description of Working

1. The system starts with a main arcade menu where the player enters their name and selects a game.
2. The menu allows users to choose between different games like Tic Tac Toe, Rock Paper Scissors, and Hangman.
3. Once a game is selected, control moves to the respective game class where the game logic runs.
4. Each game takes user input, applies game rules, and displays the result.
5. After finishing a game, the program returns to the main menu, allowing the player to select another game.
6. The system continues running until the player chooses to exit.

Arcade.java

```
1 package arcade;
2
3 import java.util.Scanner;
4
5 public class Arcade {
6
7     static Scanner sc = new Scanner(System.in); 3 usages
8
9     public static void main (String[] args) {
10
11         System.out.println("==== CONSOLE ARCADE ====");
12
13         System.out.print("Enter Player Name: ");
14         String player = sc.nextLine();
15
16         int choice;
17         char again;
18
19         do {
20
21             System.out.println("\nSelect Game");
22             System.out.println("1. Tic Tac Toe");
23             System.out.println("2. Rock Paper Scissors");
24             System.out.println("3. Hangman");
25             System.out.println("4. Exit");
26
27             System.out.print("Choice: ");
28             choice = sc.nextInt();
29
30             switch (choice) {
31                 case 1:
32                     TicTacToe.play(player);
33                     break;
34                 case 2:
35                     RockPaperScissors.play(player);
36                     break;
37                 case 3:
38                     Hangman.play(player);
39                     break;
40                 case 4:
41                     System.out.println("Thanks for playing!");
42                     return;
43                 default:
44                     System.out.println("Invalid choice");
45             }
46
47             System.out.print("\nPlay another game? (y/n): ");
48             again = sc.next().charAt(0);
49
50         } while (again == 'y' || again == 'Y');
51
52         System.out.println("Goodbye " + player + "!");
53     }
54 }
```

TicTacToe.java

```
1 package arcade;
2
3 import java.util.Scanner;
4
5 public class TicTacToe { 1 usage
6
7     public static void play (String player) { 1 usage
8
9         Scanner sc = new Scanner(System.in);
10
11         char board[][] = {
12             {' ', ' ', ' ', ' '},
13             {' ', ' ', ' ', ' '},
14             {' ', ' ', ' ', ' '}
15         };
16
17         char current = 'X';
18
19         for (int turn=0; turn<9; turn++) {
20             printBoard(board);
21
22             System.out.println("Player " + current + " enter row (0-2) and column (0-2):");
23
24             int r = sc.nextInt();
25             int c = sc.nextInt();
26
27             if (board[r][c] == ' ') {
28                 board[r][c] = current;
29
30                 if (checkWin(board, current)) {
31                     printBoard(board);
32                     System.out.println("🏆 Player " + current + " wins!");
33                     return;
34                 }
35
36                 current = (current == 'X') ? 'O' : 'X';
37             } else {
38                 System.out.println("Cell already used!");
39                 turn--;
40             }
41         }
42
43         printBoard(board);
44         System.out.println("Game Draw!");
45     }
46 }
```

```

47  static void printBoard (char b[][]) { 3 usages
48      System.out.println("\n  0  1  2");
49
50      for (int i=0; i<3; i++) {
51          System.out.print(i + " ");
52
53          for (int j=0; j<3; j++) {
54              System.out.print(b[i][j]);
55              if (j < 2)
56                  System.out.print(" | ");
57          }
58
59          System.out.println();
60
61          if (i < 2)
62              System.out.println(" ---+---+---");
63      }
64  }
65
66  static boolean checkWin (char b[][], char p) { 1 usage
67      for (int i=0; i<3; i++) {
68          if (b[i][0] == p && b[i][1] == p && b[i][2] == p)
69              return true;
70      }
71
72      for (int i=0; i<3; i++) {
73          if (b[0][i] == p && b[1][i] == p && b[2][i] == p)
74              return true;
75      }
76
77      if (b[0][0] == p && b[1][1] == p && b[2][2] == p)
78          return true;
79
80      if (b[0][2] == p && b[1][1] == p && b[2][0] == p)
81          return true;
82
83      return false;
84  }
85  }

```

RockPaperScissors.java

```
1 package arcade;
2
3 import java.util.*;
4
5 public class RockPaperScissors { 1 usage
6
7     public static void play (String player) { 1 usage
8
9         Scanner sc = new Scanner(System.in);
10        Random rand = new Random();
11
12        String options[] = {"Rock", "Paper", "Scissors"};
13
14        System.out.println("\n=== Rock Paper Scissors ===");
15        System.out.println("Choose Match Type:");
16        System.out.println("1 Best of 3");
17        System.out.println("2 Best of 5");
18
19        int rounds = (sc.nextInt() == 1) ? 3 : 5;
20
21        int userScore = 0;
22        int compScore = 0;
23
24        for (int i=1; i<=rounds; i++) {
25            System.out.println("\nRound " + i);
26
27            System.out.println("0 Rock 1 Paper 2 Scissors");
28            int user = sc.nextInt();
29
30            int comp=rand.nextInt( bound: 3);
31
32            System.out.println(player+" chose "+options[user]);
33            System.out.println("Computer chose "+options[comp]);
34
35            if (user == comp) {
36                System.out.println("Draw");
37            } else if ((user == 0 && comp == 2) || (user == 1 && comp == 0) || (user == 2 && comp == 1)) {
38                System.out.println("You win round!");
39                userScore++;
40            } else {
41                System.out.println("Computer wins round!");
42                compScore++;
43            }
44
45            System.out.println("Score: " + player + " " + userScore + " - " + compScore + " Computer");
46        }
47
48        if (userScore > compScore)
49            System.out.println("🏆 You win the match!");
50        else if (compScore > userScore)
51            System.out.println("Computer wins the match!");
52        else
53            System.out.println("Match Draw!");
54    }
55 }
```

Hangman.java

```
1 package arcade;
2
3 import java.util.*;
4
5 public class Hangman { 1 usage
6
7     public static void play (String player) { 1 usage
8
9         Scanner sc = new Scanner(System.in);
10
11         String words[] = {
12             "computer", "programming", "interface",
13             "algorithm", "developer", "variable"
14         };
15
16         String hints[] = {
17             "Machine used to run programs",
18             "Process of writing code",
19             "Connection between systems",
20             "Step-by-step problem solving",
21             "Person who writes software",
22             "Stores values in code"
23         };
24
25         Random rand = new Random();
26
27         int index = rand.nextInt(words.length);
28
29         String word = words[index];
30         String hint = hints[index];
31
32         char guessed[] = new char[word.length()];
33         Arrays.fill(guessed, val: '_');
34
35         int lives = 6;
36
37         System.out.println("\n== HANGMAN ==");
38         System.out.println("Hint: " + hint);
39
40         HashSet<Character> used = new HashSet<>();
```

```

43     while (lives > 0) {
44         System.out.println("\nWord: " + String.valueOf(guessed));
45         System.out.println("Lives: " + lives);
46         System.out.println("Used letters: " + used);
47
48         System.out.print("Guess letter: ");
49         char ch = sc.next().charAt(0);
50
51         if (used.contains(ch)) {
52             System.out.println("Already guessed!");
53             continue;
54         }
55
56         used.add(ch);
57         boolean found = false;
58
59         for (int i = 0; i < word.length(); i++) {
60             if (word.charAt(i) == ch) {
61                 guessed[i] = ch;
62                 found = true;
63             }
64         }
65
66         if (!found) {
67             lives--;
68             System.out.println("Wrong guess!");
69         }
70
71         if (word.equals(String.valueOf(guessed))) {
72             System.out.println("\n🎉 You guessed the word: " + word);
73             return;
74         }
75     }
76
77     System.out.println("\nGame Over! Word was: " + word);
78 }

```

Output:

```
===== CONSOLE ARCADE =====  
Enter Player Name: popo  
  
Select Game  
1. Tic Tac Toe  
2. Rock Paper Scissors  
3. Hangman  
4. Exit  
Choice:  
2  
  
=== Rock Paper Scissors ===  
Choose Match Type:  
1 Best of 3  
2 Best of 5  
1  
  
Round 1  
0 Rock  1 Paper  2 Scissors  
2  
popo chose Scissors  
Computer chose Paper  
You win round!  
Score: popo 1 - 0 Computer
```